Universidad del Valle de Guatemala
Facultad de Ingeniería — CC3088 Bases de Datos 1

Proyecto 2 — Diseño de Base de Datos

Modelo Relacional y Normalización

KoreGT — Tienda en línea de computadoras y accesorios

Ciclo 1, 2026

1. Descripción del dominio de negocio

Kore GT es una tienda en línea especializada en computadoras, periféricos y accesorios para computadoras y teléfonos, con una lógica de negocio comparable a la de Amazon pero enfocada en el mercado guatemalteco. La plataforma gestiona el ciclo completo de venta: catálogo de productos, carrito de compras, cotizaciones en PDF, checkout con selección de envío y método de pago, registro de órdenes y administración de inventario.

El sistema contempla tres roles de usuario: usuario público (clientes), administrador (empleados con acceso al panel de gestión de inventario y órdenes) y superadministrador (gerentes con capacidad adicional de crear y gestionar cuentas de administrador). Los proveedores principales son Intcomex y Macrosistemas.

Los métodos de envío disponibles son: Forza (Q35, 3–5 días hábiles), Cargo Expreso (Q32, 2–5 días hábiles), Guatex (Q35, 4–6 días hábiles) y Retiro en tienda (sin costo, inmediato). Los métodos de pago son: tarjeta/PayPal (pasarela PayPal), transferencia bancaria y pago contra entrega en efectivo.

Cada producto tiene un SKU autoincrementable de 6 dígitos con base en 000200. Cada envío tiene un código alfanumérico con formato GT-XXXXXX, donde XXXXXX es un número autoincrementable de 6 dígitos con base en 000001.

El control de stock se formaliza con tres atributos explícitos en la entidad productos — stock_actual (unidades disponibles en tiempo real), stock_minimo (umbral de alerta de reposición) y stock_maximo (capacidad máxima de almacenamiento por referencia). Estos tres atributos forman un grupo lógico de inventario que depende únicamente de la PK del producto, por lo que no generan ninguna violación de las formas normales.

2. Identificación de entidades y atributos (forma no normalizada — UNF)

A partir del análisis del dominio se identifican las siguientes entidades con sus atributos en estado plano (UNF), previo a cualquier proceso de normalización. Los atributos de stock se presentan ahora de forma explícita y desagregada:

Entidad	Atributos	PK	FK(s)
usuarios	id_usuario, nombre_completo, correo, contraseña_hash, teléfono, rol (enum), fecha_creación, activo, creado_por_admin_id	`id_usuario`	creado_por_admin_id → usuarios
direcciones	id_dirección, id_usuario, alias, departamento, municipio, zona, dirección_línea1, dirección_línea2, referencia, es_predeterminada	`id_dirección`	id_usuario → usuarios
proveedores	id_proveedor, nombre, contacto_nombre, contacto_correo, contacto_teléfono, país	`id_proveedor`	—

Entidad	Atributos	PK	FK(s)
categorías	id_categoría, nombre, descripción, slug, id_categoría_padre	`id_categoría`	id_categoría_padre → categorías
productos	id_producto, sku, nombre, descripción, especificaciones(JSON), precio_unitario, stock_actual, stock_minimo, stock_maximo, id_categoría, id_proveedor, activo, fecha_creación	`id_producto`	id_categoría → categorías, id_proveedor → proveedores
imágenes_producto	id_imagen, id_producto, url, orden, es_principal	`id_imagen`	id_producto → productos
métodos_envío	id_envío_método, nombre, costo, días_mínimo, días_máximo, descripción	`id_envío_método`	—
métodos_pago	id_pago_método, nombre, tipo (enum), descripción	`id_pago_método`	—
carritos	id_carrito, id_usuario, fecha_actualización	`id_carrito`	id_usuario → usuarios
carrito_items	id_carrito_item, id_carrito, id_producto, cantidad, precio_unitario_snapshot	`id_carrito_item`	id_carrito → carritos, id_producto → productos
cotizaciones	id_cotización, id_usuario, fecha_generación, total, estado (enum), ítems (multivaluado)	`id_cotización`	id_usuario → usuarios
cotización_items	id_cotización_item, id_cotización, id_producto, cantidad, precio_unitario_snapshot, subtotal	`id_cotización_item`	id_cotización → cotizaciones, id_producto → productos
órdenes	id_orden, código_orden, id_usuario, id_dirección, id_envío_método, id_pago_método, estado (enum), subtotal, costo_envío, total, fecha_orden, notas	`id_orden`	id_usuario, id_dirección, id_envío_método, id_pago_método
orden_items	id_orden_item, id_orden, id_producto, cantidad, precio_unitario_snapshot, subtotal	`id_orden_item`	id_orden → órdenes, id_producto → productos
envíos	id_envío, id_orden, código_seguimiento (GT-XXXXXX), estado_envío, fecha_despacho, fecha_entrega_estimada, fecha_entrega_real, observaciones	`id_envío`	id_orden → órdenes
pagos	id_pago, id_orden, id_pago_método, referencia_externa, monto,	`id_pago`	id_orden → órdenes, id_pago_método → métodos_pago

Entidad	Atributos	PK	FK(s)
	estado (enum), fecha_pago, comprobante_url		
movimientos_inventario	id_movimiento, id_producto, tipo (enum: entrada/venta/ajuste), cantidad, stock_antes, stock_después, referencia, id_usuario_responsable, fecha	`id_movimiento`	id_producto → productos, id_usuario_responsable → usuarios
sesiones	id_sesión, id_usuario, token_hash, ip, user_agent, creado_en, expira_en, activa	`id_sesión`	id_usuario → usuarios

3. Primera Forma Normal (1FN)

Una relación está en 1FN si todos sus atributos contienen valores atómicos, no existen grupos repetitivos ni atributos multivaluados, y existe una clave primaria definida.

Análisis de los atributos de stock en 1FN

En la versión anterior, el campo stock era un único entero que mezclaba implícitamente el concepto de stock disponible con la ausencia de umbrales de gestión. Al desagregar el control de inventario en tres atributos atómicos e independientes, cada uno con un significado distinto, el resultado sigue cumpliendo 1FN porque:

- stock_actual (INTEGER NOT NULL DEFAULT 0): valor atómico, una sola unidad de información — las unidades físicamente disponibles.
- stock_minimo (INTEGER NOT NULL DEFAULT 0): valor atómico independiente — el umbral de reposición configurado por el administrador.
- stock_maximo (INTEGER NOT NULL): valor atómico independiente — la capacidad máxima almacenable. No existe dependencia entre estos tres atributos; cada uno puede variar sin afectar a los otros.

Otras violaciones UNF corregidas para cumplir 1FN

Violación en UNF	Corrección aplicada
cotizaciones.ítems es multivaluado (lista de productos).	Tabla cotización_items con PK propia. Cada ítem ocupa una fila.
productos.especificaciones definido como JSON con sub-atributos embebidos.	Columna JSONB en PostgreSQL (valor atómico para el motor). Alternativa: tabla producto_especificaciones(id, id_producto, clave, valor).
Múltiples direcciones por usuario eran un grupo repetitivo.	Tabla direcciones con FK a usuarios.
Imágenes de producto eran grupo repetitivo.	Tabla imágenes_producto con PK y FK a productos.

4. Segunda Forma Normal (2FN)

Una relación está en 2FN si está en 1FN y todos los atributos no clave dependen completamente de la clave primaria completa. Solo aplica a tablas con clave compuesta.

Análisis de dependencias parciales — atributos de stock

Los atributos stock_actual, stock_minimo y stock_maximo residen en la tabla productos cuya PK es id_producto (clave simple surrogate). Al ser PK simple, no puede existir ninguna dependencia parcial por definición. Los tres atributos dependen completamente y exclusivamente de id_producto:

Dependencia funcional	Diagnóstico 2FN
id_producto → stock_actual	Dependencia total. El stock disponible corresponde a un producto específico. Cumple 2FN.
id_producto → stock_minimo	Dependencia total. El umbral mínimo es una política por producto. Cumple 2FN.
id_producto → stock_maximo	Dependencia total. La capacidad máxima es una política por producto. Cumple 2FN.
Todas las demás tablas (PKs simples)	No es posible dependencia parcial. Todo el esquema cumple 2FN.

5. Tercera Forma Normal (3FN)

Una relación está en 3FN si está en 2FN y ningún atributo no clave depende transitivamente de la PK a través de otro atributo no clave.

Análisis de dependencias transitivas — incluyendo atributos de stock

Tabla	Dependencia funcional	Análisis
productos	`id_producto → stock_actual`	stock_actual depende directamente de id_producto; no hay atributo intermedio. No hay transitividad. Cumple 3FN.
productos	`id_producto → stock_minimo`	stock_minimo es una política de reposición del producto, no derivada de otro atributo. Depende directamente de la PK. Cumple 3FN.
productos	`id_producto → stock_maximo`	stock_maximo es configuración de capacidad por referencia. Depende directamente de la PK. Cumple 3FN.
productos	`id_producto → id_categoria → (nombre_cat, descripcion)`	Dependencia transitiva resuelta: datos de categoría viven en tabla categorias. En productos solo FK. Cumple 3FN.
productos	`id_producto → id_proveedor →`	Dependencia transitiva resuelta: datos de proveedor en tabla proveedores. En productos solo FK. Cumple 3FN.

Tabla	Dependencia funcional	Análisis
	`(nombre_prov, contacto...)`	
órdenes	`id_orden → id_envio_metodo → (nombre, costo, días)`	Datos del método en metodos_envio. En órdenes: FK + snapshot de costo para histórico. Cumple 3FN.
órdenes	`id_orden → id_pago_metodo → (nombre, tipo)`	Datos del método en metodos_pago. Solo FK en órdenes. Cumple 3FN.
órdenes	`id_orden → id_direccion → (depto, municipio, línea1...)`	Datos en direcciones. órdenes guarda FK + direccion_snapshot inmutable. Cumple 3FN.
pagos	`id_pago → id_pago_metodo → tipo_pago`	Datos en metodos_pago. Solo FK en pagos. Cumple 3FN.
movimientos_inventario	`id_movimiento → stock_antes / stock_despues`	Estos son snapshots históricos del momento del movimiento, no derivados del stock actual del producto. No hay transitividad problemática. Cumple 3FN.

Conclusión: los tres atributos de stock no introducen ninguna dependencia transitiva. Todos dependen directa y únicamente de id_producto. El esquema completo cumple 3FN.

6. Forma Normal de Boyce-Codd (BCFN)

Una relación está en BCFN si para toda dependencia funcional no trivial X → Y, X es una superclave. BCFN es más estricta que 3FN cuando existen múltiples claves candidatas superpuestas.

Análisis BCFN — tabla productos con atributos de stock

La tabla productos tiene dos claves candidatas: CK1 = id_producto y CK2 = sku (UNIQUE). Todas las dependencias funcionales no triviales en productos tienen como determinante a CK1 o CK2:

Dependencia funcional	Determinante es superclave?
id_producto → nombre, descripcion, precio_unitario, stock_actual, stock_minimo, stock_maximo, especificaciones, activo, fecha_creacion, id_categoria, id_proveedor	Sí. id_producto = CK1 = superclave. Cumple BCFN.
sku → nombre, descripcion, precio_unitario, stock_actual, stock_minimo, stock_maximo, especificaciones, activo, fecha_creacion, id_categoria, id_proveedor	Sí. sku = CK2 = superclave. Cumple BCFN.
Todas las demás tablas (PK surrogate simple como única CK)	Toda DF no trivial tiene a la PK como determinante. Cumple BCFN trivialmente.

Conclusión: el esquema completo cumple BCFN. La incorporación de stock_actual, stock_minimo y stock_maximo no introduce ninguna violación porque los tres atributos dependen de superclaves en todos los casos.

7. Modelo relacional

Esquema final. PK: clave primaria. FK: clave foránea. UK: restricción UNIQUE.

Tabla	Esquema relacional (notación)
roles_usuario	`roles_usuario(id_rol PK, nombre UK, descripcion)`
usuarios	`usuarios(id_usuario PK, nombre_completo, correo UK, contrasena_hash, telefono, id_rol FK→roles_usuario, activo, fecha_creacion, creado_por FK→usuarios)`
sesiones	`sesiones(id_sesion PK, id_usuario FK→usuarios, token_hash UK, ip, user_agent, creado_en, expira_en, activa)`
direcciones	`direcciones(id_direccion PK, id_usuario FK→usuarios, alias, departamento, municipio, zona, direccion_linea1, direccion_linea2, referencia, es_predeterminada)`
proveedores	`proveedores(id_proveedor PK, nombre UK, contacto_nombre, contacto_correo, contacto_telefono, pais)`
categorias	`categorias(id_categoria PK, nombre, slug UK, descripcion, id_categoria_padre FK→categorias)`
productos	`productos(id_producto PK, sku UK, nombre, descripcion, especificaciones JSONB, precio_unitario, stock_actual, stock_minimo, stock_maximo, id_categoria FK→categorias, id_proveedor FK→proveedores, activo, fecha_creacion)`
imagenes_producto	`imagenes_producto(id_imagen PK, id_producto FK→productos, url, orden, es_principal)`
metodos_envio	`metodos_envio(id_envio_metodo PK, nombre UK, costo, dias_minimo, dias_maximo, descripcion)`
metodos_pago	`metodos_pago(id_pago_metodo PK, nombre UK, tipo, descripcion)`
carritos	`carritos(id_carrito PK, id_usuario FK→usuarios, fecha_actualizacion)`
carrito_items	`carrito_items(id_carrito_item PK, id_carrito FK→carritos, id_producto FK→productos, cantidad, precio_unitario_snapshot)`
cotizaciones	`cotizaciones(id_cotizacion PK, id_usuario FK→usuarios, fecha_generacion, total, estado)`
cotizacion_items	`cotizacion_items(id_cotizacion_item PK, id_cotizacion FK→cotizaciones, id_producto FK→productos, cantidad, precio_unitario_snapshot, subtotal)`
ordenes	`ordenes(id_orden PK, codigo_orden UK, id_usuario FK→usuarios, id_direccion FK→direcciones, direccion_snapshot, id_envio_metodo FK→metodos_envio, id_pago_metodo FK→metodos_pago, estado, subtotal, costo_envio, total, fecha_orden, notas)`
orden_items	`orden_items(id_orden_item PK, id_orden FK→ordenes, id_producto FK→productos, cantidad, precio_unitario_snapshot, subtotal)`

Tabla	Esquema relacional (notación)
envios	`envios(id_envio PK, id_orden FK→ordenes, codigo_seguimiento UK, estado_envio, fecha_despacho, fecha_entrega_estimada, fecha_entrega_real, observaciones)`
pagos	`pagos(id_pago PK, id_orden FK→ordenes, id_pago_metodo FK→metodos_pago, referencia_externa, monto, estado, fecha_pago, comprobante_url)`
movimientos_inventario	`movimientos_inventario(id_movimiento PK, id_producto FK→productos, tipo, cantidad, stock_antes, stock_despues, referencia, id_usuario_resp FK→usuarios, fecha)`

8. Notas de implementación

Control de stock — tres atributos en productos

El control de inventario se gestiona mediante tres columnas en la tabla productos:

- stock_actual (INTEGER NOT NULL DEFAULT 0): se decrementa automáticamente al confirmar una orden (dentro de la transacción de creación de orden) y se incrementa al registrar una entrada de inventario. Es el valor visible al cliente y al sistema de disponibilidad.
- stock_minimo (INTEGER NOT NULL DEFAULT 0): umbral configurable por el administrador. El panel administrativo alerta cuando stock_actual <= stock_minimo para disparar una reposición con el proveedor (Intcomex o Macrosistemas).
- stock_maximo (INTEGER NOT NULL): límite de almacenamiento por referencia. El sistema impide registrar entradas de inventario que harían superar este valor. Es un control de capacidad de bodega.

La tabla movimientos_inventario actúa como bitácora: registra stock_antes y stock_despues en cada operación (entrada, venta, ajuste), preservando el historial completo de cambios para auditoría y reportes, sin que esto implique redundancia de datos porque ambos valores son snapshots del momento del movimiento, no calculables retroactivamente.

Restricción CHECK en PostgreSQL

```
CONSTRAINT chk_stock_minimo CHECK (stock_minimo >= 0),
CONSTRAINT chk_stock_maximo CHECK (stock_maximo >= stock_minimo),
CONSTRAINT chk_stock_actual CHECK (stock_actual >= 0)
```

SKU de productos

El SKU se implementa en PostgreSQL como una secuencia iniciada en 200 con formato de 6 dígitos con ceros a la izquierda:

```
CREATE SEQUENCE seq_sku START 200 INCREMENT 1;
sku CHAR(6) NOT NULL DEFAULT LPAD(nextval('seq_sku')::TEXT, 6, '0')
```

Código de seguimiento de envíos (GT-XXXXXX)

El código de seguimiento se genera con una secuencia iniciada en 1 concatenando el prefijo GT-:

```
CREATE SEQUENCE seq_envio START 1 INCREMENT 1;
```

```
-- Generado en trigger INSERT ON envios:
NEW.codigo_seguimiento := 'GT-' || LPAD(nextval('seq_envio')::TEXT, 6, '0');
```

Roles de usuario como tabla de referencia

Los roles (cliente, administrador, superadministrador) se almacenan en roles_usuario en lugar de un ENUM de PostgreSQL, permitiendo flexibilidad futura sin redefinir el tipo de datos.

Snapshots de precios y dirección

precio_unitario_snapshot en orden_items y cotizacion_items, costo_envio en ordenes y direccion_snapshot preservan los valores exactos al momento de la transacción, garantizando integridad histórica inmutable.

Transacciones explícitas requeridas

- Registro de nueva orden: BEGIN → decrementar stock_actual en productos → insertar en ordenes → insertar en orden_items → insertar en pagos (estado: pendiente) → insertar en movimientos_inventario (tipo: venta) → COMMIT / ROLLBACK.
- Aprobación de pago por transferencia: BEGIN → UPDATE pagos.estado → UPDATE ordenes.estado → COMMIT.
- Entrada de inventario (admin): BEGIN → UPDATE productos SET stock_actual = stock_actual + cantidad → INSERT en movimientos_inventario (tipo: entrada) → COMMIT.
- Creación de cuenta de administrador (solo superadmin): BEGIN → INSERT en usuarios → asignar id_rol de admin → COMMIT.

Índices justificados (CREATE INDEX)

- productos(id_categoria) — Filtrado del catálogo por categoría.
- productos(id_proveedor) — Reportes de inventario por proveedor.
- productos(stock_actual) — Consultas de productos con stock bajo (stock_actual <= stock_minimo).
- carrito_items(id_carrito) — Recuperación del carrito activo.
- orden_items(id_orden) — Detalle de orden.
- ordenes(id_usuario) — Historial de compras del usuario.
- envios(codigo_seguimiento) — Búsqueda de envío por código GT-XXXXXX.
- sesiones(token_hash) — Validación de sesión en cada request autenticado.
- movimientos_inventario(id_producto, fecha) — Reportes de movimiento por producto.

9. Resumen de cumplimiento de formas normales

Tabla	1FN	2FN	3FN	BCFN
roles_usuario	✓	✓	✓	✓
usuarios	✓	✓	✓	✓
sesiones	✓	✓	✓	✓
direcciones	✓	✓	✓	✓
proveedores	✓	✓	✓	✓
categorias	✓	✓	✓	✓
productos	✓	✓	✓	✓
imagenes_producto	✓	✓	✓	✓
metodos_envio	✓	✓	✓	✓
metodos_pago	✓	✓	✓	✓
carritos	✓	✓	✓	✓
carrito_items	✓	✓	✓	✓
cotizaciones	✓	✓	✓	✓
cotizacion_items	✓	✓	✓	✓
ordenes	✓	✓	✓	✓
orden_items	✓	✓	✓	✓
envios	✓	✓	✓	✓
pagos	✓	✓	✓	✓
movimientos_inventario	✓	✓	✓	✓

El modelo relacional de Kore GT está completamente normalizado hasta BCFN. La incorporación explícita de los atributos de control de stock (stock_actual, stock_minimo, stock_maximo) en la tabla productos no introduce ninguna violación de las formas normales porque los tres atributos son atómicos, dependen completa y únicamente de la PK, y no generan dependencias transitivas entre sí ni con otros atributos.

Repositorio de github: https://github.com/turis0077/KoreGT